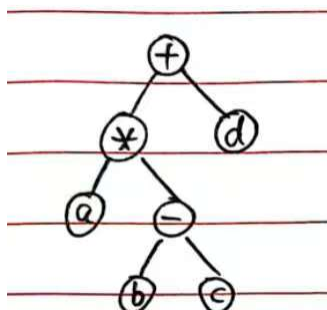


2024秋数据结构与数据库参考答案by Gnay

一、

1. **A.** 插在第一个位置移动 n , 插在最后移动0, 平均 $n/2$

2. **B.** 画树, 求后序遍历。也可以用ppt上的算法



3. **C**

4. **B.** 根据题意, 遍历时先访问节点 v , 再左子树, 最后右子树, 是先序遍历。

5. **B.** 每层都只有一个节点有孩子, 则除根节点外, 所有层都有两个节点。

6. **D.**

$H(49) = 49 \% 11 = 5$ 已有元素, 冲突。二次探测序列法 $d_i = 1^2, -1^2, 2^2, -2^2 \dots$

第一次探测: $5 + 1^2 = 6$ 已有元素, 冲突。

第二次探测: $5 - 1^2 = 4$ 已有元素, 冲突。

第三次探测: $5 + 2^2 = 9$ 未被占有, 故 $\text{add}(49)=9$ 。

7. **B.** 前序遍历按根-左-右的顺序, 后序遍历按左-右-根, 对于叶子节点, 相对次序无区别。

8. **C.**

枢轴为60, 初始序列: [60,80,55,40,42,85]。low=0,high=5

high找<60, 找到42, 填到low: [42,80,55,40,(60),85]。low=0,high=4

low找>60, 找到80, 填到high: [42,(60),55,40,80,85]。low=1,high=4

high找<60, 找到40, 填到low: [42,40,55,(60),80,85]。low=1,high=3

low找<60, 装上high, 放入枢轴: [42,40,55,60,80,85]

9. **B.** 最大即节点数, 最小时为完全二叉树, 计算 $h=8$ 时最大节点数 (满二叉树): $2^8 - 1 = 511$ 刚好为满二叉树。

10. **C.** 这种情况下插入排序只需要比较, 不需要移动, 复杂度为 $O(n)$ 。

11. **A.** $f(5)=f(3)+f(1)+1=3$, 这里调用了3次。 $f(3)=1$ 调用1次。共4次。

12. **D.** $\sum_{i=1}^{n-2} (n-i+1) = 3 + \dots + n = (n+3)(n-2)/2$

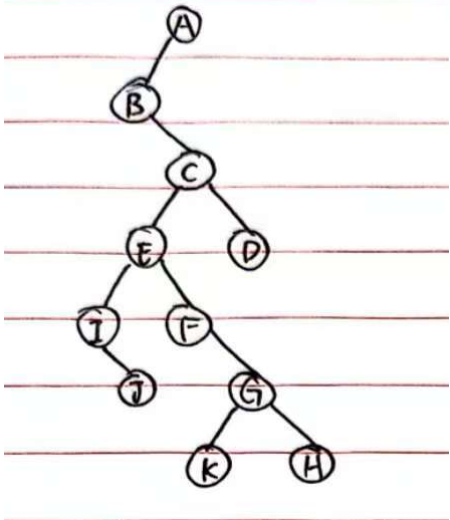
13. **C.** $\sin(n)$ 最大为1

14. **A.** 每个用于比较的关键字都被“夹”在前面几个关键字确定的区间中。A中160 $\notin [200, 450]$

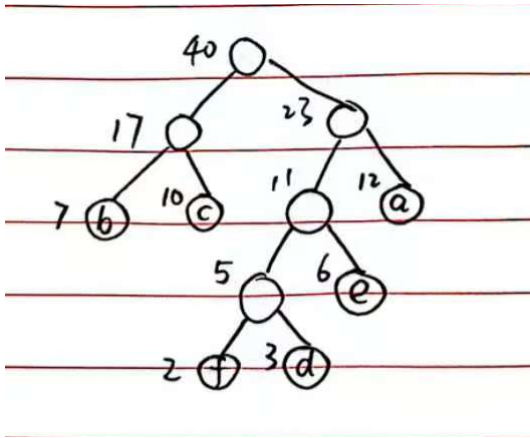
15. **B.** 增量 $d=3$ 时, 序列分组为[9,13,20],[1,7,23],[4,8,15]分别有序。

二、

1. 树如图。先序遍历：ABCEIJFGKHD，后序遍历：JIKHGFEDCBA



2. (1)如图



(2) a:11; b:00; c:01; d:1001; e:101; f:1000

(3) Total = $12 \times 2 + 7 \times 2 + 10 \times 2 + 3 \times 4 + 6 \times 3 + 2 \times 4 = 96$

3. (1)

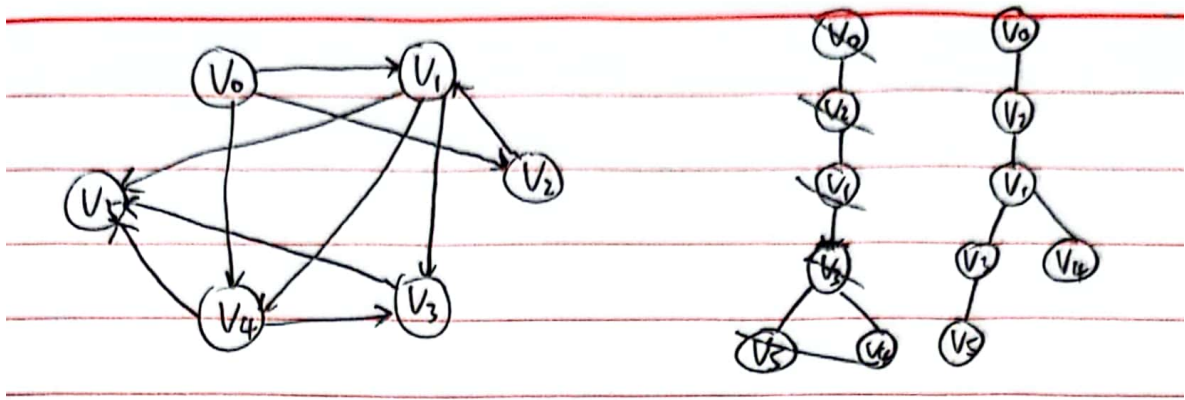
0	1	2	3	4	5	6	7	8	9	10	11	12
33	12	46	14	56	71	82	60	23	\	21	\	\

其中56,82,60,23有冲突

(2)各元素成功查找比较次数：71(1),14(1),46(1),12(1),33(1),82(2),60(3),56(4),21(1),23(8)

ASL = $23/10 = 2.3$

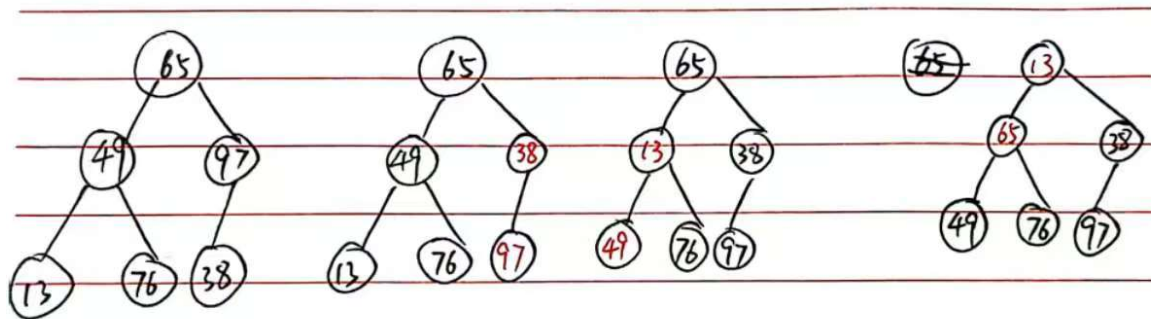
4. (1)有向图如左图所示。找入度为0的顶点，删除从它出发的边，重复此过程，共有两个可能的拓扑序列：V0, V2, V1, V4, V3, V5或V0, V2, V1, V3, V4, V5



(2)过程: $V_0 \rightarrow V_2 \rightarrow V_1 \rightarrow V_3 \rightarrow V_5 \rightarrow$ (回溯) $\rightarrow V_4$ 。故遍历序列为: **$V_0, V_2, V_1, V_3, V_5, V_4$** 。题目要求按邻接表进行dfs, 故顺序错也算错。

(3)如右图所示

5. 如图, 标红的是每次更新的节点



三、

1. // 模拟入栈出栈过程即可

```
bool isValidPopSequence(int *popSequence, int n) {
    Stack S;
    InitStack(S, n); //如果实在想不起来创建栈的形式, 用数组+top手搓一个栈问题也不大 (可能代码量还更小 (雾)
    int nextPush=0;
    int e;
    for (int i = 0; i < n; i++) {
        int target = popSequence[i];
        if (!StackEmpty(S)) GetTop(S, e);
        while ((StackEmpty(S) || e != target) && nextPush <= n) {
            Push(S, nextPush++);
            GetTop(S, e);
        }
        if (!StackEmpty(S)) {
            GetTop(S, e);
            if (e == target) Pop(S, e); else {
                DestroyStack(S);
                return 0;
            }
        }
    }
    DestroyStack(S);
    return 0;
}
```

```

    }
}
Destroy(S);
return 1;
}

```

2. // 维护两个指针，左指针扫到 ≥ 0 与右指针扫到 < 0 交换

```

void rearrangeArray(int *a, int n) {
    int i = 0, j = n - 1;
    while (i < j) {
        while (i < j && a[i] < 0) i++;
        while (i < j && a[j] >= 0) j--;
        if (i < j) {
            int t = a[i]; a[i] = a[j]; a[j] = t;
            i++; j--;
        }
    }
}
}

```

3. //法1: BFS

```

int Treewidth(TreeNode *root){
    if (!root) return 0;
    Queue Q; InitQueue(Q);
    EnQueue(Q, root);
    TreeNode *x;
    int maxW = 0;
    while (!QueueEmpty(Q)) {
        int levelCnt = QueueLength(Q);
        if (levelCnt > maxW) maxW = levelCnt;
        for (int i = 0; i < levelCnt; i++) {
            DeQueue(Q, x);
            if (x->left) EnQueue(Q, x->left);
            if (x->right) EnQueue(Q, x->right);
        }
    }
    DestroyQueue(Q);
    return maxW;
}

```

//法2: DFS

```

int cnt[1000];
void dfs(TreeNode *x, int depth){
    if (!x) return;
    cnt[depth]++;
    dfs(x->left, depth+1);
    dfs(x->right, depth+1);
}
int Treewidth(TreeNode *root){
    memset(cnt, 0, sizeof(cnt));
    dfs(root, 0);
}

```

```
int d=0,maxw=0;
while (cnt[d]){
    if (cnt[d]>maxw) maxw=cnt[d];
    d++;
}
return maxw;
}
```